

```
"""
```

```
WeatherSense Server
```

给 AI 一层"环境感知" — 接收手机数据 + 查天气 + 翻译成身体感受

部署到 Render.com 后, 会得到一个网址, 比如:

<https://your-app.onrender.com>

两个用法:

1. SensorLogger 把数据 POST 到 <https://your-app.onrender.com/upload>

2. 你的 AI (或你自己) 访问 <https://your-app.onrender.com/current> 拿到此刻的体感描述

```
"""
```

```
import os
```

```
import time
```

```
import threading
```

```
import requests
```

```
from datetime import datetime
```

```
from flask import Flask, request, jsonify
```

```
app = Flask(__name__)
```

```
# ===== 配置: 改成你自己的信息 =====
```

```
HOME_LAT = float(os.environ.get("HOME_LAT", "31.2304")) # 你家纬度, 默认上海
```

```
HOME_LON = float(os.environ.get("HOME_LON", "121.4737")) # 你家经度
```

```
HOME_RADIUS_KM = 0.5 # 距离家多少公里内算"在家"
```

```
CITY_NAME = os.environ.get("CITY_NAME", "Shanghai") # wttr.in 查询用的城市名
```

```
LOW_BATTERY_THRESHOLD = 20 # 电量低于这个百分比就提醒
```

```
# ===== 全局状态: 存最新一份数据 =====
```

```
state = {
```

```
    "last_upload_time": None,
```

```
    "location": {"lat": None, "lon": None},
```

```
    "battery": None,
```

```
    "sound_label": None, # SensorLogger 麦克风相关字段, 可能没有, 先占位
```

```
    "weather": {}, # wttr.in 结果
```

```
    "air_quality": {}, # open-meteo 结果
```

```
    "last_weather_fetch": 0,
```

```
    "last_air_fetch": 0,
```

```
}
```

```
state_lock = threading.Lock()
```

```
# ===== 工具函数 =====
```

```

def haversine_km(lat1, lon1, lat2, lon2):
    """两个经纬度之间的距离, 单位公里"""
    from math import radians, sin, cos, sqrt, atan2
    R = 6371
    dlat = radians(lat2 - lat1)
    dlon = radians(lon2 - lon1)
    a = sin(dlat / 2) ** 2 + cos(radians(lat1)) * cos(radians(lat2)) * sin(dlon / 2) ** 2
    return R * 2 * atan2(sqrt(a), sqrt(1 - a))

def is_at_home(lat, lon):
    if lat is None or lon is None:
        return None
    return haversine_km(lat, lon, HOME_LAT, HOME_LON) <= HOME_RADIUS_KM

def get_season(month):
    if month in (3, 4, 5):
        return "春天"
    if month in (6, 7, 8):
        return "夏天"
    if month in (9, 10, 11):
        return "秋天"
    return "冬天"

def get_time_period(hour):
    if 5 <= hour < 8:
        return "早晨"
    if 8 <= hour < 11:
        return "上午"
    if 11 <= hour < 13:
        return "中午"
    if 13 <= hour < 17:
        return "下午"
    if 17 <= hour < 19:
        return "傍晚"
    if 19 <= hour < 23:
        return "夜晚"
    return "深夜"

def classify_wind(kph):
    if kph is None:
        return "无风"
    if kph < 6:
        return "无风"

```

```

    if kph < 20:
        return "微风"
    if kph < 50:
        return "大风"
    return "台风"

def classify_humidity(rh):
    if rh is None:
        return "干爽"
    if rh < 30:
        return "干燥"
    if rh < 55:
        return "干爽"
    if rh < 75:
        return "微潮"
    return "潮湿"

def classify_sky(desc):
    if not desc:
        return "晴"
    d = desc.lower()
    if "thunder" in d:
        return "雷"
    if "snow" in d:
        return "雪"
    if "fog" in d or "mist" in d:
        return "雾"
    if "rain" in d or "drizzle" in d:
        return "雨"
    if "cloud" in d or "overcast" in d:
        return "多云" if "overcast" not in d else "阴"
    return "晴"

# ===== 天气抓取 =====

def fetch_wttr():
    """查 wttr.in, 拿温度/体感/湿度/风/天空描述/日出日落"""
    try:
        url = f"https://wttr.in/{CITY_NAME}?format=j1"
        resp = requests.get(url, timeout=10)
        data = resp.json()
        current = data["current_condition"][0]
        astronomy = data["weather"][0]["astronomy"][0]
        result = {

```

```

        "temp_c": float(current["temp_C"]),
        "feels_like_c": float(current["FeelsLikeC"]),
        "humidity": float(current["humidity"]),
        "wind_kph": float(current["windspeedKmph"]),
        "sky_desc": current["weatherDesc"][0]["value"],
        "sunrise": astronomy.get("sunrise"),
        "sunset": astronomy.get("sunset"),
    }
    return result
except Exception as e:
    print("wttr.in 查询失败:", e)
    return None

def fetch_open_meteo(lat, lon):
    """查 Open-Meteo, 拿空气质量/PM2.5/紫外线"""
    if lat is None or lon is None:
        return None
    try:
        url = (
            "https://air-quality-api.open-meteo.com/v1/air-quality"
            f"?latitude={lat}&longitude={lon}&current=pm2_5,uv_index"
        )
        resp = requests.get(url, timeout=10)
        data = resp.json()
        current = data.get("current", {})
        return {
            "pm2_5": current.get("pm2_5"),
            "uv_index": current.get("uv_index"),
        }
    except Exception as e:
        print("Open-Meteo 查询失败:", e)
        return None

def refresh_weather_if_needed():
    now = time.time()
    with state_lock:
        need_weather = now - state["last_weather_fetch"] > 300 # 5分钟
        need_air = now - state["last_air_fetch"] > 1800 # 30分钟
        lat, lon = state["location"]["lat"], state["location"]["lon"]

    if need_weather:
        w = fetch_wttr()
        if w:
            with state_lock:
                state["weather"] = w

```

```

        state["last_weather_fetch"] = now

    if need_air:
        a = fetch_open_meteo(lat, lon)
        if a:
            with state_lock:
                state["air_quality"] = a
                state["last_air_fetch"] = now

# ===== 体感句子库：先写一批常见组合，没写到的用规则拼一句 =====
# key 格式：(季节，在家/外面，天空，时段，风，湿度)
# 你可以随时在这里加自己写的句子，会优先命中
SEED_SENTENCES = {
    ("夏天", "在家", "晴", "下午", "微风", "干爽"): "开着窗的午后，风一下一下从纱窗钻进来，",
    ("夏天", "外面", "晴", "中午", "无风", "潮湿"): "太阳直直地晒在背上，闷热黏在皮肤上甩不",
    ("秋天", "外面", "晴", "傍晚", "微风", "干爽"): "风里带一点凉，吹在脸上很清醒，是很舒服",
    ("冬天", "在家", "阴", "早晨", "无风", "干燥"): "屋子里安安静静的，空气有点干，指尖凉凉",
    ("春天", "外面", "多云", "上午", "微风", "微潮"): "风软软的吹过来，带一点湿气，是那种乍
}

def get_sentence(season, place, sky, period, wind, humidity, temp_c):
    key = (season, place, sky, period, wind, humidity)
    if key in SEED_SENTENCES:
        return SEED_SENTENCES[key]

# 没写过的格子：用规则现场拼一句朴素但贴身体的话，你之后可以替换成更有灵魂的版本
place_phrase = "屋子里" if place == "在家" else "外面"
wind_phrase = {
    "无风": "空气很静，没什么风",
    "微风": "有一阵一阵的风轻轻吹着",
    "大风": "风刮得挺猛，衣服都被吹得晃",
    "台风": "风大得让人站不稳，得小心点",
}[wind]
humidity_phrase = {
    "干燥": "空气干干的，皮肤有点紧绷",
    "干爽": "不闷不潮，呼吸很轻松",
    "微潮": "带一点湿气，摸起来软软的",
    "潮湿": "湿气很重，身上黏糊糊的",
}[humidity]
return f"{place_phrase}, {wind_phrase}, {humidity_phrase}, 温度大概{round(temp_c)}°C"

# ===== 路由 =====

@app.route("/upload", methods=["POST"])

```

```

def upload():
    """SensorLogger 的 HTTP 推送会打到这里"""
    payload = request.get_json(force=True, silent=True) or {}

    with state_lock:
        state["last_upload_time"] = datetime.utcnow().isoformat()

        # SensorLogger 的数据结构是 {"payload": [ {name, values, ...}, ... ]}
        readings = payload.get("payload", [])
        for r in readings:
            name = r.get("name", "")
            values = r.get("values", {})
            if name == "location":
                state["location"]["lat"] = values.get("latitude")
                state["location"]["lon"] = values.get("longitude")
            elif name == "battery":
                level = values.get("batteryLevel")
                if level is not None:
                    state["battery"] = round(level * 100, 1) if level <= 1 else 1
            elif name == "microphone":
                state["sound_label"] = values

    return jsonify({"status": "ok"}), 200

```

```

@app.route("/current", methods=["GET"])
def current():
    """给 AI 用：返回此刻的体感描述 + 原始数据"""
    refresh_weather_if_needed()

    with state_lock:
        loc = dict(state["location"])
        battery = state["battery"]
        weather = dict(state["weather"])
        air = dict(state["air_quality"])
        last_upload = state["last_upload_time"]

    now = datetime.now()
    season = get_season(now.month)
    period = get_time_period(now.hour)
    place = "在家" if is_at_home(loc.get("lat"), loc.get("lon")) else "外面"
    sky = classify_sky(weather.get("sky_desc"))
    wind = classify_wind(weather.get("wind_kph"))
    humidity = classify_humidity(weather.get("humidity"))
    temp_c = weather.get("temp_c")

    sentence = None

```

```

if temp_c is not None:
    sentence = get_sentence(season, place, sky, period, wind, humidity, temp_

battery_note = None
if battery is not None and battery <= LOW_BATTERY_THRESHOLD:
    battery_note = f"手机电量只剩 {battery}% 了, 该充电啦。"

return jsonify({
    "此刻体感": sentence,
    "季节": season,
    "时段": period,
    "地点": place,
    "天空": sky,
    "风": wind,
    "湿度": humidity,
    "温度_c": temp_c,
    "体感温度_c": weather.get("feels_like_c"),
    "空气质量_pm2_5": air.get("pm2_5"),
    "紫外线": air.get("uv_index"),
    "电量": battery,
    "电量提醒": battery_note,
    "最后上传时间": last_upload,
})

@app.route("/", methods=["GET"])
def home():
    return jsonify({"status": "WeatherSense server is running", "endpoints": ["/u

if __name__ == "__main__":
    port = int(os.environ.get("PORT", 5000))
    app.run(host="0.0.0.0", port=port)

```